

Ejercicio 2 - Búsqueda informada

Rutas en México con A*

En este ejercicio agregué la opción de buscar una ruta entre dos ciudades y verla en el mapa. Adapté el A* de Rumania en `find_route.py` para usarlo desde la terminal y en `route_search.js` para ejecutarlo en el navegador. También añadí los selectores de origen y destino, el botón para calcular la ruta y el panel con los resultados.

Decidí que el mapa hiciera la búsqueda directamente en JavaScript para poder abrir el HTML con doble clic. Así no hace falta tener un servidor de Python corriendo. Las dos versiones siguen las mismas reglas, aunque en Python usé un heap para obtener la opción con menor f y en JavaScript una lista ordenada. Para este grafo de 1,000 ciudades, la lista permite mantener el código sencillo y funciona bien.

Usé el id como estado de búsqueda porque el nombre de una ciudad puede repetirse. En la interfaz dejé visibles la ciudad y el estado; si ambos coinciden, aparecen las coordenadas para distinguirlas. En la terminal, un nombre repetido muestra las opciones y pide indicar el estado o usar el id. Así el programa trabaja con una ciudad exacta, aunque el usuario no vea su identificador en el mapa.

Cuando se eligen el origen y el destino, el programa consulta las conexiones que cargó del JSON. Las organicé como una lista de vecinos para cada ciudad, considerando ambos sentidos de cada conexión. La búsqueda empieza con el origen y revisa sus vecinos. Para cada uno calcula cuánto costaría llegar hasta él, sumando lo recorrido y los kilómetros de esa conexión.

Si esa llegada mejora lo que ya se conocía y la ciudad todavía no fue expandida, se calcula su estimación hasta el destino. A^* suma los kilómetros recorridos, g , y esa estimación, h , para obtener $f = g + h$. Las opciones quedan pendientes en la frontera y se procesa primero la que tenga menor f . Las otras se conservan, así que el algoritmo puede volver a considerar otra rama del camino. Para los empates mantuve el mismo orden de inserción en ambas versiones y ordené los vecinos por id.

Para h usé haversine, que calcula la distancia directa al destino con las coordenadas. Esa distancia no supera un recorrido formado por varios tramos de la misma medida, por lo que sirve como una heurística admisible. Como los kilómetros del JSON están redondeados, apliqué un pequeño ajuste para mantener esa propiedad. La estimación se calcula cuando hace falta agregar una opción; no preparo una tabla completa para todas las ciudades al iniciar cada búsqueda.

Para controlar las repeticiones guardé el mejor costo conocido de cada ciudad en `best_g` y las ciudades ya expandidas en `explored`. Si encuentro una llegada más barata a una ciudad pendiente, actualizo su costo y agrego la nueva opción. La ciudad se marca como explorada al procesarla, no al descubrirla. Las entradas viejas se ignoran cuando salen de la frontera si esa ciudad ya fue expandida. Como la heurística que usé es consistente, no hace falta reabrir ciudades ya procesadas. Estos datos solo se mantienen en memoria durante la búsqueda.

Cuando el destino sale de la frontera como la mejor opción, reconstruyo la ruta usando el padre guardado en cada nodo, es decir, la ciudad desde la que llegué a él. Sigo esos padres hasta el origen y después invierto el orden. Con esa lista resalto los nodos y las conexiones del camino y muestro los kilómetros, los tramos y los nodos expandidos. Cada nueva consulta empieza con sus propios costos y opciones pendientes.

Probé Tijuana a Cancún y Mexico City a Monterrey. En la ruta larga obtuve 4528.20 km, 124 tramos y 949 nodos expandidos. La terminal y el mapa dieron los mismos resultados, y también comparé los costos con Dijkstra para comprobar que fueran mínimos. La ruta larga me sirvió para ver que, aunque la heurística orienta la búsqueda, A^* todavía puede necesitar explorar una parte grande del grafo antes de llegar al destino.

Ruta	Costo	Tramos	Nodos expandidos
Tijuana → Cancún	4528.20 km	124	949
Mexico City → Monterrey	1041.87 km	27	428

Evidencias: Tijuana → Cancún

En esta ruta obtuve 4528.20 km, 124 tramos y 949 nodos expandidos. Las dos capturas muestran el mismo resultado en la terminal y en el mapa.

Ejecución en CMD

```
C:\Users\david\Documents\IA-FMAT-2026\busqueda informada\ejercicio-02\Mexico map>python find_route.py
--from-city Tijuana --to Cancún
Algorithm: A* (f = g + h)
From: Tijuana, Baja California [id=6]
To: Cancún, Quintana Roo [id=25]
Heuristic: haversine al destino x 0.998517792 (ajuste por redondeo del JSON)
Status: success
Path: Tijuana -> Villa del Prado 2da Sección -> Terrazas del Valle -> Tecate -> Progreso -> ... (125
ciudades en total) ... -> Peto -> Felipe Carrillo Puerto -> Tulum -> Playa del Carmen -> Cancún
Depth: 124 hops
Cost: 4528.20 km
Expanded: 949 nodes
```

Ruta mostrada en el mapa



Evidencias: Mexico City → Monterrey

Para la segunda ruta obtuve 1041.87 km, 27 tramos y 428 nodos expandidos. El costo también coincide en ambas ejecuciones.

Ejecución en CMD

```
C:\Users\david\Documents\IA-FMAT-2026\busqueda informada\ejercicio-02\Mexico map>python find_route.py
--from-city "Mexico City" --to Monterrey
Algorithm: A* (f = g + h)
From: Mexico City, Mexico City [id=0]
To: Monterrey, Nuevo León [id=10]
Heuristic: haversine al destino x 0.998517792 (ajuste por redondeo del JSON)
Status: success
Path: Mexico City -> Gustavo Adolfo Madero -> Puerto Escondido (Tepeolulco Puerto Escondido) -> Coacalco -> San Pablo de las Salinas -> ... (28 ciudades en total) ... -> Camargo -> China -> Cadereyta Ji-ménez -> Jardines de la Silla (Jardines) -> Monterrey
Depth: 27 hops
Cost: 1041.87 km
Expanded: 428 nodes
```

Ruta mostrada en el mapa

